অধ্যায় ২০

JavaScript Security Fundamentals, Web Application Security এবং Secure Coding Practices

২০.১ Learning Objectives

এই অধ্যায় শেষে তুমি Threat Modeling, Trust Boundary, XSS, CSRF, Injection, Authentication বনাম Authorization, Browser Storage, CORS, CSP এবং Secure Coding-এর মৌলিক সিদ্ধান্ত নিতে পারবে। একই সঙ্গে SecureTrip Booking Portal-এ Input Validation, Safe DOM Rendering ও Controlled Data Flow প্রয়োগ করবে।

২০.২ Security মানে কী?

একটি Application সঠিকভাবে কাজ করলেই তা নিরাপদ হয় না। Security হলো **Confidentiality** (অননুমোদিত ব্যক্তি তথ্য না পাওয়া), **Integrity** (তথ্য অনুমতি ছাড়া বদলানো না যাওয়া), এবং **Availability** (বৈধ ব্যবহারকারীর জন্য সেবা চালু থাকা)—এই তিনটির ভারসাম্য।

উদাহরণ: একটি Booking Form-এ Client থেকে `total: 1` পাঠিয়ে যদি ৫০,০০০ টাকার Booking তৈরি করা যায়, তাহলে Server Client-এর মূল্যকে বিশ্বাস করেছে। এটি Integrity Failure। Server-কে নিজস্ব Package Catalog থেকে মূল্য গণনা করতে হবে।

Security principle: Browser-এ থাকা Data, Hidden Input, Local Storage, URL Parameter ও JavaScript State—সবই আক্রমণকারী পরিবর্তন করতে পারে। তাই Browser কখনো Security Authority নয়।

২০.৩ Threat Modeling ও Trust Boundary

একটি Feature লিখতে যাওয়ার আগে চারটি প্রশ্ন করো:

1. কোন Asset রক্ষা করতে হবে? (User Data, Session, Booking, Payment)
2. কে এই Asset ব্যবহার করতে পারে? (Guest, Customer, Staff, Admin)
3. Data কোথা থেকে আসছে? (Form, URL, API, Database, Third-party)
4. কোন Boundary পেরিয়ে Data Trust পাবে? (Browser → API → Database)

```
Untrusted Browser
   |  JSON Request (untrusted)
   v
HTTPS + Server API
   |  Validate + Authenticate + Authorize
   v
Business Rules (price, capacity, status)
   |  Parameterized Queries / Transactions
   v
Database
```

এই Diagram-এ Validation, Authentication এবং Authorization আলাদা দায়িত্ব। শুধু একটি Login Button থাকলেই User অন্যের Booking দেখতে পারবে না—এমন নিশ্চয়তা পাওয়া যায় না।

২০.৪ XSS — Cross-Site Scripting

XSS হয় যখন Untrusted Content Browser-এ Executable Markup/Script হিসেবে ব্যবহৃত হয়। নিচের Code বিপজ্জনক:

```javascript
// বিপজ্জনক: input-এর HTML browser parse করবে
result.innerHTML = userProvidedName;
```

সাধারণ Text দেখানোর জন্য নিরাপদ পদ্ধতি:

```javascript
const item = document.createElement('li');
item.textContent = userProvidedName;
list.append(item);
```

`textContent` HTML Interpret করে না। কিন্তু `textContent`-এর ব্যবহার সব Context-এ নিরাপত্তার বিকল্প নয়: URL, CSS, HTML Attribute এবং Script Context-এর আলাদা Rules আছে। HTML Rich Text সত্যিই প্রয়োজন হলে বিশ্বস্ত Sanitization Library এবং Context-aware Output Encoding ব্যবহার করতে হবে।

URL সতর্কতা: `a.href = untrustedUrl` করার আগে `new URL()` দিয়ে Scheme (`https:`/`http:`), Domain Allowlist এবং Business Requirement যাচাই করো। `javascript:` URL অনুমোদন করা যাবে না।

২০.৫ Input Validation বনাম Output Encoding

Validation পরীক্ষা করে Data ব্যবসার নিয়ম মানে কি না। Output Encoding নিশ্চিত করে Data যেখানে দেখানো হচ্ছে সেখানে Executable Code হয়ে না যায়। দুটি আলাদা কাজ।

```javascript
function validateTravelers(value) {
  const n = Number(value);
  if (!Number.isSafeInteger(n) || n < 1 || n > 8) {
    throw new Error('Travelers must be 1–8');
  }
  return n;
}
```

Validation **Client ও Server উভয় জায়গায়** থাকতে পারে, কিন্তু Server-side Validation বাধ্যতামূলক। Browser Validation ব্যবহারকারীর অভিজ্ঞতা উন্নত করে; Security Boundary তৈরি করে না।

২০.৬ SQL Injection ও Parameterized Queries

Untrusted String দিয়ে SQL Query তৈরি করা বিপজ্জনক:

```javascript
// করবেন না
const query = `SELECT * FROM bookings WHERE email = '${email}'`;
```

Parameterized Query-তে Query Structure এবং Data আলাদা থাকে:

```javascript
// PostgreSQL node-postgres example (requires configured server and pg package)
const result = await pool.query(
  'SELECT id, status FROM bookings WHERE email = $1',
  [email]
);
```

Database Permission-ও Least Privilege নীতিতে সীমিত রাখতে হবে। Parameterization Data Query Injection ঠেকায়; এটি Authorization বা Rate Limiting-এর বিকল্প নয়।

২০.৭ Authentication বনাম Authorization

Authentication: কে Request করছে? **Authorization:** তার এই Resource-এ Access আছে কি?

ধরা যাক `GET /api/bookings/123`। শুধু Session Valid কি না দেখলে যথেষ্ট নয়। Server-কে যাচাই করতে হবে Booking `123` এই User-এর কি না অথবা তার উপযুক্ত Staff Permission আছে কি না। একে Object-level Authorization বলা হয়।

```javascript
// Conceptual server-side example
if (!session?.userId) return reply.status(401).send();
const booking = await bookings.findById(request.params.id);
if (!booking) return reply.status(404).send();
if (booking.ownerId !== session.userId && !session.roles.includes('admin')) {
  return reply.status(403).send();
}
return reply.send(booking);
```

এই Snippet Framework-নির্ভর Pseudocode; প্রকৃত Framework অনুযায়ী Syntax ও Error Handling সমন্বয় করতে হবে।

২০.৮ Password, Sessions ও Cookies

Password কখনো Plaintext-এ Store করবে না। Server-side Password Hashing-এর জন্য Argon2id (উপযুক্ত Parameters সহ) বা সুপ্রতিষ্ঠিত Password Hashing Scheme ব্যবহার করতে হবে। General-purpose `SHA-256(password)` Password Storage-এর যথেষ্ট সমাধান নয়।

Session Cookie-এর সাধারণ Production Flags:

```http
Set-Cookie: session=<opaque-session-id>; HttpOnly; Secure; SameSite=Lax; Path=/
```

`HttpOnly` JavaScript থেকে Cookie পড়া কঠিন করে, `Secure` HTTPS-only Delivery নির্দেশ করে, `SameSite` Cross-site Request-এর কিছু ঝুঁকি কমায়। এগুলো **CSRF প্রতিরোধের সম্পূর্ণ বিকল্প নয়**। State-changing Request-এ CSRF Token বা নির্ভরযোগ্য Origin Checking প্রয়োজন হতে পারে।

Sensitive Authentication Token Local Storage-এ রাখার ঝুঁকি বুঝতে হবে: XSS থাকলে JavaScript তা পড়তে পারে। Threat Model অনুযায়ী Session Architecture নির্বাচন করতে হবে।

২০.৯ CSRF, CORS এবং Same-Origin Policy

CSRF: Victim-এর Browser দিয়ে Victim-এর Authentication Context-এ অনাকাঙ্ক্ষিত Request করানো। **CORS:** Browser কোন Origin-এর Script অন্য Origin-এর Response পড়তে পারবে তার Policy। **CORS কোনো Authentication বা CSRF Defense নয়।**

```javascript
// Browser request: cookie-backed session হলে CSRF strategy লাগবে
await fetch('/api/bookings', {
  method: 'POST',
  credentials: 'same-origin',
  headers: {'Content-Type': 'application/json', 'X-CSRF-Token': csrfToken},
  body: JSON.stringify(payload)
});
```

`csrfToken` Server-issued এবং Session-bound হওয়া উচিত। Header নাম দিলেই নিরাপত্তা তৈরি হয় না—Server-কে Token যাচাই করতে হবে।

২০.১০ CSP, HTTPS এবং Security Headers

Content Security Policy (CSP) XSS-এর Impact কমাতে সাহায্য করতে পারে। উদাহরণ:

```http
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
```

এটি একটি প্রাথমিক Example, সবার জন্য Copy-Paste Production Policy নয়। Script Loading, Third-party Service, Inline Script, Nonce/Hash এবং Browser Support যাচাই করতে হবে। HTTPS/TLS, Secure Cookie, HSTS Deployment ও Dependency Updates-ও গুরুত্বপূর্ণ।

২০.১১ Secrets, Dependencies এবং Logging

API Key, Database Password ও Signing Secret Frontend Bundle-এ রাখা যাবে না। `.env` ফাইল Commit না করে Secret Manager বা Server Environment-এ রাখো। Logs-এ Password, Full Card Number, Session Token, Authorization Header বা Sensitive Personal Data রাখা যাবে না।

Dependencies Update ও Vulnerability Audit করো, কিন্তু Automated Scanner-এর Output মানবিক যাচাই ছাড়া Blindly Fix করবে না। Third-party Script যত কম, Supply-chain Attack Surface তত কম।

২০.১২ Practical Project — SecureTrip Booking Portal

Goal: একটি Browser-based Booking Request Demo তৈরি করা, যাতে Form Validation, Safe Text Rendering, Fixed Price Catalog, Unique Request ID এবং Summary থাকবে। এটি Payment, Authentication, Real Reservation বা Secure Server System নয়।

File Structure

```
source-code/chapter-20/
├── index.html
├── style.css
├── booking-core.mjs
├── app.mjs
└── booking-core.test.mjs
```

Run: Folder-এ `python -m http.server 8000` চালিয়ে `http://localhost:8000` খুলতে পারো। Project Folder Root থেকে Serve করলে URL হবে `/source-code/chapter-20/`। JavaScript Module File Browser-এ `file://` দিয়ে খুললে কিছু Browser-এ CORS Restriction হতে পারে।

Tests: `node --test source-code/chapter-20/booking-core.test.mjs` (Node.js 20+ recommended)।

Core Logic

```javascript
export function validateBooking(input) {
  // বাস্তব Implementation: booking-core.mjs
  // 1. Name/Email validate
  // 2. Catalog থেকে Package খুঁজে বের করো
  // 3. Travelers 1–8 integer কিনা দেখো
  // 4. Price = trustedCatalogPrice * travelers
  // 5. Consent যাচাই করো
}
```

এটি Architecture Outline; পূর্ণ Runnable Implementation Companion Source Code-এর `source-code/chapter-20/booking-core.mjs`-এ রয়েছে। UI-তে প্রতিটি Booking-এর Text `textContent` দিয়ে দেখানো হয়েছে, `innerHTML` দিয়ে নয়।

Security Boundary Checklist

Control	Demo Status	Production Requirement
Client validation	আছে	Server revalidation বাধ্যতামূলক
Catalog-derived price	Client demo-তে আছে	Server authoritative pricing
Safe DOM text rendering	আছে	Contextual encoding everywhere
Booking ID	Browser crypto random	Server-generated ID and uniqueness
Authentication	নেই	Secure identity/session design
Authorization	নেই	Per-resource access checks
Persistence	In-memory only	Database and transactions
CSRF / Rate limit	নেই	Threat-based server controls
Payments	নেই	Certified payment provider integration

২০.১৩ Guided Security Review

1. `booking-core.mjs`-এ `total` Field Client থেকে নেওয়া হয়েছে কি? কেন?
2. UI-তে `innerHTML` ব্যবহার করলে কোন ধরনের ঝুঁকি তৈরি হতে পারে?
3. `crypto.getRandomValues()` দিয়ে Browser ID তৈরি হলেও Server কেন Unique Constraint চাইবে?
4. Session Cookie `HttpOnly` হলেও CSRF কেন ঘটতে পারে?
5. `CORS: *` Authentication-এর বিকল্প নয় কেন?
6. Booking List যদি Server-এ থাকে, কীভাবে Object-level Authorization নিশ্চিত করবে?

২০.১৪ Exercises — ১০টি

১. Name Input-এর Length Validation ২–৮০ অক্ষর কেন, ব্যাখ্যা করো।

২. User-controlled HTML `textContent` দিয়ে Render করো।

৩. Travelers `2.5` দিলে Reject করার Function লেখো।

৪. Client-supplied `price: 1` উপেক্ষা করে Catalog Price Calculate করো।

৫. Safe Redirect-এর জন্য Relative Path Validation করো।

৬. Authorization এবং Authentication-এর মধ্যে দুটি পার্থক্য লেখো।

৭. SQL Query Parameterization-এর Example লেখো।

৮. CSRF Token ছাড়া Cookie-based POST Request-এর ঝুঁকি ব্যাখ্যা করো।

৯. Login Error Log থেকে Sensitive Fields বাদ দেওয়ার Strategy লেখো।

১০. SecureTrip-এর জন্য Server-side Deployment Checklist তৈরি করো।

Sample Solutions

```javascript
// 1 — Length rule (trim + Unicode normalization আগে করো)
const validNameLength = name => name.length >= 2 && name.length <= 80;

// 2 — Safe DOM text
const span = document.createElement('span');
span.textContent = userText;

// 3 — Integer range
const validTravelers = n => Number.isSafeInteger(Number(n)) && Number(n) >= 1 && Number(n) <= 8;

// 4 — Authoritative price lookup (server-side-ও একই নিয়ম)
const total = catalog.find(p => p.id === packageId).price * travelers;

// 5 — Simplified safe return path (full version: booking-core.mjs)
const safePath = path => typeof path === 'string' && path.startsWith('/') &&
!path.startsWith('//') ? path : '/';
```

৬. Authentication Identity যাচাই করে, Authorization Permission যাচাই করে। ৭. `pool.query('SELECT * FROM bookings WHERE id = $1', [id])`। ৮. Cookie Automatic Attach হতে পারে; Server-side CSRF Defense দরকার। ৯. Structured Log Allowlist ব্যবহার করো; Secret Fields Redact করো। ১০. HTTPS, Server Validation, AuthN/AuthZ, CSRF, Rate Limits, Parameterized SQL, DB Constraints, Secret Management, Monitoring, Dependency Audit এবং Backup/Recovery Plan অন্তর্ভুক্ত করো।

সতর্কতা: ৫ নম্বর Simplified Snippet Production-ready URL Validation নয়; Test করা পূর্ণ Function Source File-এ দেখো।

২০.১৫ MCQ — ১০টি

১. Browser Input-কে কেন Untrusted ধরা হয়? A) Slow B) User can modify C) Unicode D) CSS

২. সাধারণ Text Output-এর জন্য কোনটি ভালো? A) `innerHTML` B) `eval()` C) `textContent` D) `document.write()`

৩. SQL Injection কমাতে কী ব্যবহার করা উচিত? A) String Concatenation B) Parameterized Queries C) Hidden Fields D) CSS

৪. Authorization কী যাচাই করে? A) Identity B) Permission C) Password Hash D) Browser Version

৫. `HttpOnly` Cookie-এর মূল কাজ? A) Encrypt Database B) Restrict JS access C) Prevent all CSRF D) Disable HTTPS

৬. CORS কি Authentication-এর বিকল্প? A) হ্যাঁ B) না C) Only Safari D) Only Node

৭. Price কে Authoritatively Calculate করবে? A) Browser B) Server C) CSS D) URL

৮. Password Storage-এর জন্য কী উপযুক্ত? A) Plaintext B) Fast SHA-256 only C) Argon2id D) Base64

৯. CSP-এর ভূমিকা কী? A) Reduce certain injection impacts B) Replace Validation C) Replace HTTPS D) Replace Login

১০. SecureTrip Demo-তে কী নেই? A) Safe DOM rendering B) Catalog pricing C) Real authentication D) Input validation

Answers & Explanations

1. **B** — Browser State User বদলাতে পারে।
2. **C** — `textContent` HTML Parse করে না।
3. **B** — Query Structure ও Data আলাদা রাখে।
4. **B** — Resource Permission যাচাই করে।
5. **B** — JavaScript থেকে Cookie পড়ার সুযোগ সীমিত করে।
6. **B** — CORS Browser Read Policy, Identity Check নয়।
7. **B** — Server authoritative pricing নিশ্চিত করে।
8. **C** — Argon2id Password Hashing-এর উপযোগী।
9. **A** — CSP Defense-in-depth Layer।
10. **C** — Demo-তে Real Authentication নেই।

২০.১৬ Chapter Recap

Security একটি Feature নয়; এটি Architecture, Coding, Deployment ও Monitoring-এর সমন্বিত প্রক্রিয়া। Client Validation, Safe DOM Rendering এবং Trusted Price Calculation-এর ধারণা SecureTrip Demo-তে প্রয়োগ করেছি। Production Security-এর জন্য Server-side Validation, Authentication, Authorization, Session Defense এবং Database Controls অপরিহার্য। পরবর্তী অধ্যায়ে Design Patterns ও Scalable Architecture আলোচনা করব।